

Hypertext Preprocessor (PHP)

Wykłady 5

Bartosz Lauks

10 kwietnia 2026



- ➊ Wprowadzenie
- ➋ Formularze z PHP
- ➌ Materiały i dokumentacja

- 1 Wprowadzenie
- 2 Formularze z PHP
- 3 Materiały i dokumentacja

Wprowadzenie

Czym jest PHP ?

- **PHP (Hypertext Preprocessor)** to popularny, skryptowy język programowania, zaprojektowany głównie do tworzenia dynamicznych stron internetowych i aplikacji webowych.
- Jest to język wykonywany po stronie serwera (**server-side**), co oznacza, że kod **PHP** jest przetwarzany na **serwerze**, a użytkownik otrzymuje już gotowy, przetworzony dokument HTML.
- PHP umożliwia interakcję z różnymi zasobami systemowymi, takimi jak:
 - Obsługuje systemy bazodanowe.
 - Operacje na plikach (tworzenie, odczyt, zapis, usuwanie).
 - Komunikacja z systemem operacyjnym, może wykonywać polecenia systemowe.

Zastosowania PHP

Gdzie używa się PHP?

- Tworzenie dynamicznych stron internetowych
- Budowa aplikacji webowych (backend)
- Systemy CMS (np. WordPress)
- Frameworki MVC (np. Laravel, Symfony)
- API i usługi REST
- Systemy e-commerce (np. Magento, PrestaShop)

Wprowadzenie

Historia wersja 1 i 2

- **PHP** powstał w roku **1993** jako program **CGI (Common Gateway Interface)** w **C**, czyli interfejs umożliwiający serwerowi wykonywanie zewnętrznych programów.
- Twórcą języka jest **Rasmus Lerdorf**, których używał go do utrzymywania swojej osobistej strony domowej. Twórca zaimplementował także obsługę formularzy internetowych i komunikację z bazami danych , nazywając tę implementację **PHP/FI** jako akronim od "**Personal Home Page/Forms Interpreter**".

Wprowadzenie

Historia wersja 3 i 4

- W roku **1997 Zeev Suraski** i **Andi Gutmans** przepisali **parser** tworząc podwaliny pod wersję **3.0**, równocześnie zmieniając nazwę języka na rekurencyjny akronim **PHP: Hypertext Preprocessor**.
- Suraski i Gutmans rozpoczęli następnie ponowne przepisywanie rdzenia PHP, tworząc w **1999** roku **Zend Engine**, który jest narzędziem wykonującym kod **PHP**.
- W **2000** roku wydano **4.0** wersję języka, opartą na owym silniku.
- Kluczowe zmiany to:
 - Poprawiona obsługa sesji.
 - Wsparcie dla większej liczby serwerów webowych.
 - Lepsza obsługa buforowania outputu.
 - Większa stabilność i bezpieczeństwo.

Wprowadzenie

Historia wersja 5 i 6

- **PHP 5** był dużym krokiem naprzód dzięki wprowadzeniu **Zend Engine 2.0** i wsparcia dla programowania obiektowego (**OPP**).
- Nowe funkcje obejmowały:
 - Obsługę wyjątków (try-catch).
 - Wprowadzenie PDO (PHP Data Objects) dla bezpieczniejszego dostępu do baz danych.
 - Rozszerzoną obsługę XML (SimpleXML, DOM, XSLT).
 - Poprawione mechanizmy zarządzania pamięcią i wydajnością.
- **PHP 6** był ambitnym projektem, który miał wprowadzić pełne wsparcie dla **Unicode**. Niestety, implementacja wprowadzała duże problemy wydajnościowe, co spowodowało porzucenie.

Wprowadzenie

Historia wersji 7

- **PHP 7** to jedna z najważniejszych wersji języka PHP, która została oficjalnie wydana pod koniec **2015** roku.
- Była to pierwsza duża aktualizacja po PHP 5.6 i przyniosła znaczące zmiany w wydajności, składni oraz bezpieczeństwie.
- Wersja siódma jest do tej pory najpopularniejszą wersją języka wykorzystywaną w aplikacjach.
- Główne cele wprowadzenia PHP 7 to:
 - Znacząca poprawa wydajności, dzięki nowemu silnikowi **Zend Engine 3.0**.
 - Lepsza optymalizacja pamięci, co pozwala obsługiwać więcej żądań przy mniejszym zużyciu zasobów.
 - Nowe funkcje i możliwości w zakresie programowania obiektowego i typowania.
 - Poprawa bezpieczeństwa, eliminacja przestarzałych funkcji i zwiększenie stabilności kodu.

Wprowadzenie

Historia wersji 8

- **PHP 8** to kolejna duża ewolucja języka, przynosząca szereg nowoczesnych funkcji i dalsze poprawy wydajności.
- Najważniejszą nowością jest **JIT (Just-In-Time Compiler)**, który znacząco poprawia szybkość wykonywania kodu.
- Dzięki tej technice PHP jest bezpośrednio przetwarzany do kodu maszynowego.
- W starszych wersjach kod PHP był przetwarzany do opcode'ów, czyli niskopoziomowej reprezentacji skryptu, a następnie wykonywany.

Wprowadzenie

PHP a żądanie strony WWW

- Aby zrozumieć rolę PHP w przetwarzaniu żądania strony WWW, należy przyjrzeć się procesowi obsługi żądania HTTP, komunikacji między klientem (np. przeglądarką) a serwerem oraz sposobowi wykonywania kodu PHP.
- Załóżmy że wysyłamy żądanie HTTP pod adres URL strony WWW, która zwraca nam dokładną datę na serwerze.
- Każde odwiedzenie strony internetowej składa się z kilku kluczowych etapów:
 - **Przeglądarka wysyła żądanie HTTP** do serwera WWW, wskazując żądany zasób.
 - **Serwer WWW odbiera żądanie** i przekazuje je do interpretera PHP.
 - **PHP interpretuje kod**, generuje wynik w postaci HTML i zwraca go do serwera WWW
 - **Serwer WWW odsyła wygenerowany HTML** do przeglądarki użytkownika.
 - **Przeglądarka renderuje stronę** i wyświetla ją użytkownikowi.



◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ 🔍 ↺

Wprowadzenie

Wykonywanie PHP oraz jego składnia

- Oczywiście jeśli chcemy korzystać z PHP musimy go zainstalować na serwerze WWW. Często zdarza się w systemach Linux, że język ten jest już dostępny wraz z domyślnie instalowanym serwerem HTTP na urządzeniu.
- W naszym przypadku nie musimy się tym przejmować ponieważ narzędzie **XAMPP** zawiera w sobie najnowszą stabilną wersję PHP.
- Jeżeli chcemy, aby strona internetowa była przetwarzana przez **pre-procesor** musimy zmieść jej rozszerzenie na [.php](#).
- W pliku musimy zadeklarować **tag** otwierający `<?php` i tag zamykający `?>`
- Wszystko co znajduje się między nimi, jest traktowane jako kod PHP.

Listing 1: Przykład strony WWW z PHP. [Przykład localhost/lecture5/index.php](http://localhost/lecture5/index.php)

```
1 <!DOCTYPE html>
2 <html lang="pl">
3     <head>
4         <meta charset="UTF-8">
5         <meta name="viewport" content="width=device-width,
6         initial-scale=1.0">
7         <title>Czas serwera</title>
8     </head>
9     <body>
10        <h1>Aktualny czas serwera</h1>
11        <p>
12            <?php
13                echo date("Y-m-d H:i:s");
14            ?>
15        </p>
16    </body>
17 </html>
```

Wprowadzenie

Funkcja echo, print i date()

- W powyższym przykładzie poznajemy jedną z podstawowych funkcji PHP **echo**. Służy do wyświetlania danych.
- W PHP istnieje również **print()**, której wynikiem jest także wyświetlania danych, ale dodatkowo funkcja ta zwraca wartość **1**, więc może być użyte w wyrażeniach.
- Zaleca się wykorzystywanie **echo** do wyświetlania danych, ponieważ jest szybsze od **print()**.
- Kolejna funkcja **date()** zwraca sformatowany znacznik (**timestamp**) czasu **Unix**. Jako argument przyjmie format, w jakim zostanie przedstawiony czas.

Wprowadzenie

Wynik odpowiedzi serwera

- Przyjrzyjmy się wynikowi HTML, jaki zwrócił nam serwer, gdy wykonaliśmy żądanie storny WWW zawierającej kod PHP.
localhost/lecture5/index.php
- Nie ma tam deklaracji tagów ani kodu, co potwierdza, że PHP wykonuje się po stronie serwera.
- W ten sposób możemy zobaczyć, że skrypt jest nie widoczny dla użytkownika (klienta).
- Kod PHP nigdy nie opuszcza serwera. W przeciwieństwie do JS, HTML i CSS, które są wykonywane lub interpretowane przez urządzenie użytkownika (przeglądarkę).

Wprowadzenie

Informacja o PHP oraz jego konfiguracji

- Pełną specyfikację używanego przez serwer PHP-a oraz jego całej specyfikacji takiej jak wykorzystywane moduły pliki konfiguracyjne itd. możemy uzyskać za pomocą **phpinfo()**
Przykład : <localhost/lecture5/phpInfo.php>
- W informacji tam zawartych znajdziemy lokalizację pliku **php.ini** to główny plik konfiguracyjny dla języka PHP, który kontroluje różne aspekty jego działania na serwerze. Przykładowe Podstawowe dyrektywy:
 - **memory_limit** - Określa maksymalną ilość pamięci, którą może używać skrypt PHP.
 - **max_execution_time** - Definiuje maksymalny czas (w sekundach), przez który może działać skrypt PHP.
 - **upload_max_filesize** - Ustawia maksymalny rozmiar pliku, który może zostać przesłany za pomocą formularzy HTML.

Wprowadzenie

Błędy w PHP

- Pomimo tego, że kod PHP wykonuje się po stronie serwera, co zabezpiecza logikę biznesową aplikacji przed wglądem użytkownika, może dojść do wyświetlenia fragmentów kodu, nazw plików oraz numerów linii, jeśli PHP napotka błąd.
- Dlatego programista powinien zadbać o odpowiednią obsługę błędów oraz konfigurację środowiska tak, aby aplikacja nie ujawniała wrażliwych informacji i nie generowała niekontrolowanych komunikatów błędów w środowisku produkcyjnym.
- Tak jest w przypadku XAMPP, który domyślnie ma wyłączone wyświetlanie błędów.

Wprowadzenie

Konfiguracja php.ini

- Na potrzeby dewelopowania aplikacji należy zmodyfikować ustawienia odpowiadające za wyświetlanie błędów w pliku [/etc/php.ini](#):
 - **display_errors = On**
 - **display_startup_errors = On**
 - **error_reporting = E_ALL**
- Jeśli wprowadzimy jakiegolwiek zmiany w konfiguracji, należy zrestartować serwer HTTP powiązany z PHP.
- Poniższy program powinien wyświetlić błąd PHP.

[localhost/lecture5/phpError.php](#)

Wprowadzenie

Komentarze

- Komentarz w kodzie PHP to fragment, który nie jest wykonywany przez program, a jego jedynym celem jest ułatwienie zrozumienia kodu osobom go przeglądającym.
- W PHP można stosować kilka rodzajów komentarzy:
 - **Jednolinijkowe komentarze**- oznaczane są za pomocą `//` lub `#`.
 - **Wielolinijkowe komentarze** - Oznaczane są za pomocą `/* ... */`.
- W PHP istnieją też **Komentarze w stylu PHPDoc**, które są zastąpione w wersji 8 przez **atrybuty**. Oba stosowane są do dokumentacji kodu i mogą zawierać znaczniki dla generatorów dokumentacji.

Przykład: localhost/lecture5/comments.php

Wprowadzenie

Zmienne

- **Zmienna** w PHP to zasób, który przechowuje wartość (np. liczbę, tekst, tablicę, obiekt itp.).
- W PHP zmienne są dynamiczne, co oznacza, że nie musisz deklarować ich typu przed przypisaniem wartości.
- PHP automatycznie przypisuje typ zmiennej na podstawie jej wartości.
- Zmienne w PHP zawsze zaczynają się od znaku dolara \$, a następnie nazwy zmiennej. Na przykład:

Listing 2: Przykład

```
1 $zmienna = 10; // zmienna typu integer
2 $tekst = "Hello, World!"; // zmienna typu string
```

Wprowadzenie

Typy zmiennych w PHP

- PHP obsługuje kilka podstawowych typów zmiennych:
 - **Integer (int):** Liczby całkowite, np. 5, -3, 0.
 - **Float (double):** Liczby zmiennoprzecinkowe, np. 3.14, -0.5, 2.0.
 - **String:** Ciągi znaków, np. "Hello", 'PHP'.
 - **Boolean (bool):** Wartości logiczne, które mogą być true lub false.
 - **Array:** Tablice, które mogą przechowywać wiele wartości \$tablica = [1, 2, 3].
 - **Object:** Obiekty klasy w PHP.
 - **NULL:** Specjalna zmienna, która reprezentuje brak wartości.
- W PHP możesz używać **stałych** za pomocą słowa kluczowego **const** lub funkcji **define()**.
- Stałe są specjalnym rodzajem zmiennych, które nie mogą być zmieniane po ich zdefiniowaniu.

Przykład: localhost/lecture5/virable.php

Wprowadzenie

Tablice

- **Tablice** w PHP to jeden z podstawowych typów danych, który pozwala przechowywać wiele wartości w jednej zmiennej.
- Tablica to kolekcja elementów, które mogą być różnego typu, a jej elementy mogą być dostępne za pomocą indeksów lub kluczy (tablice asocjacyjne).

Przykład: localhost/lecture5/array.php

Wprowadzenie

Debugowanie kodu w PHP

- W PHP, `print_r()` i `var_dump()` to funkcje wykorzystywane do wyświetlania informacji o zmiennych, które pomagają w procesie debugowania kodu.
- Oto krótkie omówienie każdej z nich:
 - Funkcja `print_r()` jest używana do wyświetlania wartości zmiennych w formie czytelnej dla człowieka. Jest to przydatne, gdy chcemy szybko sprawdzić zawartość tablicy lub obiektu.
 - Funkcja `var_dump()` jest bardziej szczegółowa niż poprzednia. Zawiera pełne informacje o typie zmiennej oraz jej wartości. Jest szczególnie przydatna do debugowania, gdy musimy poznać typ zmiennej oraz jej zawartość.

Przykład: localhost/lecture5/debug.php

- 1 Wprowadzenie
- 2 Formularze z PHP
- 3 Materiały i dokumentacja

Formularze

Obsługa formularzy przez PHP

- W PHP, obsługa formularzy HTML jest procesem, w którym dane wprowadzone przez użytkownika w formularzu są przekazywane do skryptu PHP, który je przetwarza, weryfikuje.
- Formularze opisywaliśmy już w wcześniejszym wykładzie, ale nie przyglądaliśmy się procesowi odbioru formularza po stronie serwera i ewt. wyświetleniem wyniku.
- Użytkownik po wypełnieniu formularza i kliknięciu przycisku „Kup teraz” (submit). Wysyła dane przy użyciu metody HTTP **POST**.
- Jest ona określana na podstawie atrybutu **method="POST"**. Oznacza to, że dane zostaną przesłane w ciele żądania (payload).

Omawiany Formularz: localhost/lecture5/formularz.php

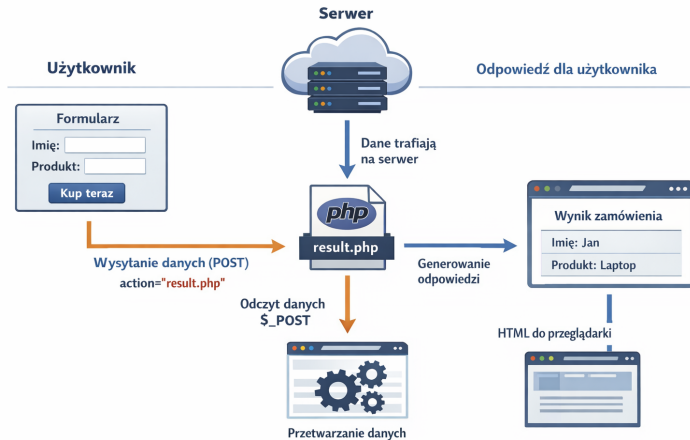
Formularze

Obsługa formularzy

- Kolejnym ważnym atrybutem jest `action="result.php"`, to właśnie on określa adres **URL**, do którego zostaną wysłane dane formularza po jego przesłaniu.
- Warto sprawdzić narzędzia deweloperskie w przeglądarce w sekcji Network, gdzie zobaczymy tam wysłane przez nas żądanie.
- Istotnym elementem w **Request Headers** jest **content-type: application/x-www-form-urlencoded**.
- To jeden z najczęściej używanych typów kodowania danych w formularzach HTML.
- Dzięki temu informujemy serwer HTTP, że przesyłamy do niego formularz z odpowiednim kodowaniem.

Obsługa formularzy przez PHP

- Teraz następuje przechwytywanie danych w pliku [result.php](#).
- Skrypt sprawdza, czy formularz został wysłany metodą **POST**, co jest warunkiem koniecznym do przetworzenia danych.
- Danę są pobierane za pomocą **zmiennej superglobalnej PHP** `$_POST`, skrypt przechwytuje dane z formularza.
- Po przechwyceniu danych, skrypt PHP generuje odpowiedź.
- W odpowiedzi tworzona jest strona HTML, która wyświetla dane w tabeli.
- Ostatnim etapem istotnym dla formularzy jest wysłanie odpowiedzi do przeglądarki użytkownika.



Rysunek 2: Wysyłanie i przetwarzanie formularza przez PHP

Formularze

Czym są Zmienne superglobalne ?

- W PHP istnieje również grupa specjalnych zmiennych, które są dostępne globalnie w całym skrypcie.
- Przykłady superglobalnych zmiennych to:
 - **\$_GET** – Zmienna, która przechowuje dane przesyłane metodą GET. Dane widoczne w URL (np. [?id=5](#))
 - **\$_POST** – Zmienna, która przechowuje dane przesyłane metodą POST. Dane niewidoczne w URL, przesyłane w body żądania.
 - **\$_SESSION** – Zmienna przechowująca dane sesji użytkownika.
 - **\$_COOKIE** – Zmienna przechowująca dane z ciasteczek (cookies).
 - **\$_SERVER** – Zmienna, która zawiera informacje o serwerze i środowisku wykonawczym.
 - **\$_FILES** – Zmienna zawierająca dane o przesyłanych plikach.
 - **\$_REQUEST** – Zmienna, która zawiera dane zarówno z **\$_GET**, **\$_POST**, jak i **\$_COOKIE**.

Metody GET i POST w formularzach

Różnica między formularzem GET a POST

- Na pierwszym wykładzie omawialiśmy różnice między metodami HTTP GET oraz POST. Metoda GET, zgodnie ze standardem, służy do pobierania danych, natomiast POST do ich przesyłania oraz tworzenia nowych zasobów na serwerze.
- W powyższym przykładzie tworzyliśmy nowe zamówienie, dlatego użyliśmy metody POST. Często jednak spotykamy formularze, np. wyszukiwarki, które jedynie zwracają dane i w takich przypadkach stosuje się metodę GET.
- Dodatkową różnicą jest to, że żądanie GET nie posiada ciała (body). W związku z tym wszystkie dane są przesyłane w adresie URL w postaci **query params** (np. [?name=Jan&surname=Kowalski](#)).
- Dane przesyłane metodą GET nie są ukryte oraz mogą być zapisywane w historii przeglądarki.

Metody GET i POST w formularzach

Formularz z metodą GET

- Specyfikacja HTML pozwala na użycie metody GET w formularzach poprzez zadeklarowanie jej w atrybucie **method**.
- To wystarczy, aby przeglądarka automatycznie wysłała żądanie (request), dodając dane formularza do adresu URL w postaci parametrów.
- Należy jednak pamiętać o odpowiedniej modyfikacji skryptu PHP, który będzie te dane obsługiwał.
- Aby pobrać dane z **query parameters**, należy użyć superglobalnej tablicy **\$_GET**.

localhost/lecture5/formularzGET.php

- 1 Wprowadzenie
- 2 Formularze z PHP
- 3 **Materiały i dokumentacja**

Materiały i dokumentacja

Materiały i dokumentacja

- [PHP i MySQL. Tworzenie stron WWW. Vademecum profesjonalisty](#)
Autorzy: Luke Welling, Laura Thomson
Wydanie: 2021 (5. edycja)
Kompleksowy podręcznik do tworzenia aplikacji webowych z użyciem PHP i MySQL. Obejmuje podstawy oraz integrację z bazą danych.
- [PHP Cookbook: Modern Code Solutions for Professional Developers](#)
Autor: Eric A. Mann
Wydanie: 2023
Zbiór praktycznych rozwiązań dla programistów PHP. Skupia się na nowoczesnym PHP (8.x), bezpieczeństwie i wydajności.

Materiały i dokumentacja

Materiały i dokumentacja

- [Learning PHP, MySQL & JavaScript](#)

Autor: Robin Nixon

Wydanie: 2021 (6. edycja)

Praktyczne wprowadzenie do tworzenia dynamicznych aplikacji webowych (PHP + MySQL + JavaScript).

- www.w3schools.com/php/ - jeden z najlepszych kursów do PHP.
- php.net — oficjalna dokumentacja PHP, funkcje, przykłady, komentarze użytkowników
- php-fig.org/psr — standardy PHP (PSR), dobre praktyki programistyczne

Dziękuję za uwagę !

Bartosz Lauks